01

PARTIE 1 • RAPPEL ET CONSOLIDATION

Les structures simples et les structures de données

Ce que tu sauras faire à la fin

- Reconnaître le type d'une donnée et choisir le type qui convient à un problème
- Utiliser les opérateurs arithmétiques, relationnels et logiques sans te tromper de priorité
- Manipuler une chaîne de caractères : longueur, position, découpage, conversion
- Déclarer un vecteur, accéder à une case et modifier son contenu
- Traduire une déclaration algorithmique en Python et inversement

Pré-requis

- Notion de variable et d'affectation
- Lire / Écrire en algorithmique
- Les quatre opérations

Mots-clés

type · entier · réel · booléen · chaîne · vecteur · indice · ASCII

Durée conseillée : 3 séances

I Les types de données

1. Le type entier (int)

DÉFINITION

Le type **entier** désigne un sous-ensemble des nombres entiers relatifs $\mathbb{Z}$. Une variable entière occupe une case mémoire et ne peut contenir ni virgule ni texte.

Opérations arithmétiques et relationnelles

Algorithme	Python	Rôle	Exemple
+	+	Addition	`5 + 2 = 7`
−	-	Soustraction	`5 − 2 = 3`
*	*	Multiplication	`5 * 2 = 10`
/	/	Division	`5 / 2 = 2.5`
DIV	//	Division entière	`5 // 2 = 2`
MOD	%	Reste de la division	`5 % 2 = 1`
x^n	**	Puissance	`5 ** 2 = 25`
=	==	Égalité	`5 == 5 → True`
<	<	Inférieur	`2 < 5 → True`
>	>	Supérieur	`5 > 2 → True`
≤	<=	Inférieur ou égal	`2 <= 5 → True`
≥	>=	Supérieur ou égal	`5 >= 2 → True`
≠	!=	Différent	`5 != 2 → True`
`n ∈ [5,20]`	`5<=n<=20`	Appartenance	`5<=3<=20 → False`

ERREUR FRÉQUENTE

Ne confonds jamais = et == en Python. = **affecte** une valeur (`x = 5`), == **compare** (`x == 5`). En langage algorithmique, l'affectation s'écrit `x ← 5`.

2. Le type réel (float)

DÉFINITION

Le type **réel** désigne un sous-ensemble des nombres réels $\mathbb{R}$. En Python le séparateur décimal est le **point**, jamais la virgule : `3.14` et non `3,14`.

REMARQUE

Les réels acceptent les mêmes opérations que les entiers **sauf DIV et MOD**.

Fonctions prédéfinies sur les réels

Algorithme	Python	Rôle	À compléter
`Ent(x)`	`int(x)`	Partie entière	`int(3.51)` = *3*
`Arrondi(x)`	`round(x)`	Arrondi	`round(3.49)` = *3* `round(3.51)` = *4*
`Abs(x)`	`abs(x)`	Valeur absolue	`abs(−3)` = *3*
`RacineCarré(x)`	`sqrt(x)`	Racine carrée	`sqrt(25)` = *5.0*
`Aléa()`	`random()`	Réel aléatoire de [0 ; 1[	—
`Aléa(a,b)`	`uniform(a,b)`	Réel aléatoire de [a ; b]	—
`Aléa(m,n)`	`randint(m,n)`	Entier aléatoire de [m ; n]	—

ASTUCE

Ces fonctions ne sont pas disponibles toutes seules : il faut importer les bibliothèques.
`from math import sqrt` · `from random import random, uniform, randint`

3. Le type booléen (bool)

DÉFINITION

Le type **booléen** ne comporte que deux valeurs : **Vrai** et **Faux** (`True` et `False` en Python, avec une majuscule).

Algorithme	Python	Rôle
NON	`not`	Négation
OU	`or`	Disjonction — vrai si au moins l'un des deux est vrai
ET	`and`	Conjonction — vrai si les deux sont vrais

ACTIVITÉ — TABLE DE VÉRITÉ

Complète le tableau suivant.

x	y	NON(x)	x ET y	x OU y
V	V	*F*	*V*	*V*
V	F	*F*	*F*	*V*
F	V	*V*	*F*	*V*
F	F	*V*	*F*	*F*

ACTIVITÉ — RÉSULTAT ET TYPE

Donne le résultat et le type de chaque instruction.

Instruction	Résultat	Type
`A = 5 + 7`	*12*	*int*
`B = 5 - 7`	*-2*	*int*
`C = 5 * 7`	*35*	*int*
`D = 5 / 7`	*0.7142857142857143*	*float*
`E = 5 // 7`	*0*	*int*
`F = 5 % 7`	*5*	*int*
`G = 5 > 7`	*False*	*bool*
`H = 5 != 7`	*True*	*bool*
`I = (2>7) and (-2<5)`	*False*	*bool*
`J = (randint(1,3)<10) or (4>2)`	*True*	*bool*
`K = not (10 != -2)`	*False*	*bool*

4. Le type caractère

DÉFINITION

Un **caractère** (chiffre, lettre ou symbole) s'écrit entre guillemets : `"A"`, `"a"`, `"+"`. Une variable de type caractère contient **un caractère et un seul**. L'espace `" "` est le caractère blanc.

REMARQUE

Les caractères sont **ordonnés selon leur code ASCII**. Ainsi `"A" < "B"` est vrai, et `"a" > "b"` est faux. Attention : toutes les majuscules (65 à 90) viennent **avant** toutes les minuscules (97 à 122).

Algorithme	Python	Rôle	À compléter
`ORD(c)`	`ord(c)`	Code ASCII d'un caractère	`ord("A") = 65` `ord("E") =` *69* `ord("a") = 97` `ord("d") =` *100*
`CHR(n)`	`chr(n)`	Caractère d'un code ASCII	`chr(65) =` *'A'* `chr(66) =` *'B'* `chr(97) =` *'a'* `chr(ord("E")) =` *'E'*

ASTUCE

Pour passer d'une minuscule à sa majuscule : `chr(ord(c) - 32)`. Dans l'autre sens : `chr(ord(c) + 32)`. L'écart est toujours de 32.

5. Le type chaîne de caractères (str)

DÉFINITION

Une **chaîne de caractères** est une suite de n caractères (n ≥ 0) écrite entre guillemets. Si n = 0 la chaîne est dite **vide** et se note `""`.

Accéder aux caractères

On accède au $i^{ème}$ caractère d'une chaîne `Ch` par `Ch[i]`, avec i variant de **0** à **long(Ch) − 1**.

DÉCOUPAGE (SLICING)

```
Ch = "Bonjour aline"

Ch[0]     →  "B"
Ch[1]     →  "o"
Ch[:3]    →  "Bon"           # les 3 premiers caractères
Ch[3:]    →  "jour aline"    # tout sauf les 3 premiers
Ch[3:7]   →  "jour"          # de l'indice 3 à 7-1 = 6
Ch[-1]    →  "e"             # le dernier caractère
```

ERREUR FRÉQUENTE

Dans `Ch[d:f]`, la position **f est exclue**. `"Bonjour"[0:3]` renvoie `"Bon"` et non `"Bonj"`. C'est la source n°1 de fautes aux devoirs.

Fonctions prédéfinies sur les chaînes

Algorithme	Python	Description
`Long(ch)`	`len(ch)`	Nombre de caractères de ch
`Pos(ch1, ch2)`	`ch2.find(ch1)`	Première position de ch1 dans ch2, ou **−1** si absente
`Convch(x)`	`str(x)`	Convertit un nombre en chaîne
`Estnum(ch)`	`ch.isnumeric()`	Vrai si ch est convertible en numérique
`Valeur(ch)`	`int(ch) / float(ch)`	Convertit une chaîne en nombre (entier ou réel)
`Sous_chaine(ch,d,f)`	`ch[d:f]`	Partie de ch de la position d jusqu'à f exclue
`Effacer(ch,d,f)`	`ch[:d]+ch[f:]`	Supprime les caractères de d à f exclue
`Majus(ch)`	`ch.upper()`	Convertit ch en majuscules
—	`ch1 + ch2`	Concaténation de deux chaînes

ACTIVITÉ — CHAÎNES

Sachant que `ch = "Bonjour"`, complète :

Expression	Résultat	Expression	Résultat
`len(ch)`	*7*	`ch.find("on")`	*1*
`str(15.75)`	*'15.75'*	`"13".isnumeric()`	*True*
`"13ab".isnumeric()`	*False*	`int("12")`	*12*
`float("13.5")`	*13.5*	`int("rd85")`	*ValueError*
`ch[0:3]`	*'Bon'*	`ch[:3]+ch[7:]`	*'Bon'*
`ch.upper()`	*'BONJOUR'*	`ch[::-1]`	*'ruojnoB'*

II Le tableau à une dimension (vecteur)

DÉFINITION

Un **vecteur** est une structure de données permettant de ranger un nombre fini d'éléments **de même type**. Il est caractérisé par son **nom**, sa **taille** et le **type** de ses éléments.

Déclaration

LANGAGE ALGORITHMIQUE	IMPLÉMENTATION EN PYTHON

```
TDO
 Objet   Type / Nature
   T     tableau de 20 entiers

— ou bien, avec un type nommé —

TDNT
              Type
 Tab = tableau de 20 entiers
TDO
 Objet   Type / Nature
   T     Tab
```

```
from numpy import array

T = array([int] * 20)


# numpy doit être installé :
# pip install numpy
```

Accès aux éléments

L'accès à chaque élément se fait par un **indice**, qui commence toujours à **0**.

T	10.50	0.25	6.00	-5.00	32.00	569.00	14.00	11.00	43.00	12.00
indice	0	1	2	3	4	5	6	7	8	9

`T[0]` = *10.50* `T[1]` = *0.25* `T[9]` = *12.00*

ACTIVITÉ — MODIFIER UN VECTEUR

Soit T un vecteur de 5 entiers. Donne le contenu de chaque case après l'exécution de la séquence suivante.

LANGAGE ALGORITHMIQUE	IMPLÉMENTATION EN PYTHON

```
T[0] ← 20
T[1] ← 2
T[2] ← T[0] DIV T[1]
T[3] ← T[2] * 5
T[4] ← T[3] + T[2] * T[1]
```

```
T[0] = 20
T[1] = 2
T[2] = T[0] // T[1]
T[3] = T[2] * 5
T[4] = T[3] + T[2] * T[1]
```

T	[0]	[1]	[2]	[3]	[4]
valeur	*20*	*2*	*10*	*50*	*70*

Fiche mémo — Structures simples et données

Les 5 types simples	entier (int) · réel (float) · booléen (bool) · caractère · chaîne (str)
Affectation	x ← 5 en algorithmique, `x = 5` en Python
Division	`/` donne un réel · `//` (DIV) donne le quotient entier · `%` (MOD) donne le reste
Indices	Une chaîne et un vecteur commencent à l'indice **0** et finissent à **long − 1**
Slicing	`ch[d:f]` : f est **exclue**
ASCII	`ord(c)` → code · `chr(n)` → caractère · écart majuscule/minuscule = 32
Vecteur	Éléments de **même type**, taille fixe, accès par indice

SÉRIE D'EXERCICES 1

Structures simples et structures de données

8 exercices classés par difficulté · Facile Moyen Défi

Exercice 1 Produit et somme des chiffres — Facile

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de lire un entier N puis d'afficher la somme `Som` et le produit `Prod` de ses chiffres.

EXEMPLE D'EXÉCUTION

```
Donner N : 84
Som = 12 et Prod = 32
```

Exercice 2 Inversion d'un entier — Facile

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de lire un entier N composé de quatre chiffres, puis d'afficher R composé des mêmes chiffres écrits à l'envers.

EXEMPLE D'EXÉCUTION

```
Donner N : 5124
R = 4215
```

Exercice 3 Nombre valide parfait — Moyen

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de déterminer si un entier N de deux chiffres est **valide parfait**.

N est valide parfait s'il est multiple de chacun des deux chiffres qui le composent.

EXEMPLE D'EXÉCUTION

```
Donner N : 24
24 est valide parfait
```

Exercice 4 Nombre de chiffres — Facile

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de déterminer et d'afficher le nombre de chiffres NC d'un entier N.

EXEMPLE D'EXÉCUTION

```
Donner N : 25468
Nombre de chiffres = 5
```

Exercice 5 Du binaire au décimal — Moyen

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de lire un nombre binaire B formé de quatre chiffres, puis de calculer et d'afficher le nombre décimal D correspondant.

Rappel : $b_3b_2b_1b_0$ vaut $b_3\times2^3 + b_2\times2^2 + b_1\times2^1 + b_0\times2^0$.

EXEMPLE D'EXÉCUTION

```
Donner B : 1001
1001 en binaire = 9 en décimal
```

Exercice 6 Permuter deux caractères

Moyen

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de lire une chaîne ch, de permuter son premier et son dernier caractère, puis d'afficher le résultat.

EXEMPLE D'EXÉCUTION

```
Donner ch : "bonjour"
ch = "ronjoub"
```

Exercice 7 Extraire les initiales

Moyen

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de saisir une chaîne CH formée d'un nom et d'un prénom séparés par un espace, et d'afficher la chaîne S formée des deux initiales en majuscules.

EXEMPLE D'EXÉCUTION

```
Donner CH : "mouhamed ali"
S = "MA"
```

Exercice 8 Somme des codes ASCII

Défi

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de saisir une chaîne CH et d'afficher la somme des codes ASCII de tous ses caractères.

EXEMPLE D'EXÉCUTION

```
Donner CH : "Bac"
Somme = 66 + 97 + 99 = 262
```